House Segmentation from Aerial Images

SEG4180 - University of Ottawa

Oussama Touahri

April 6th, 2026

1 Introduction

This project/lab implements an end-to-end machine learning pipeline for house segmentation using aerial imagery. The objective is to detect and segment building regions at the pixel level. The pipeline includes dataset preparation, model training, evaluation, and deployment via a REST API with Docker and CI/CD integration.

2 Dataset and Preprocessing

The dataset used is a satellite building segmentation dataset containing aerial images and corresponding bounding box annotations.

Since segmentation masks were not directly available, pixel-level masks were generated from bounding boxes. Each bounding box was converted into a binary mask where building regions are labeled as 1 and background as 0.

The dataset was split into training, validation, and test sets. Images were resized to 256×256 resolution for efficient training.

3 Model Architecture

A U-Net architecture was used for the segmentation task. U-Net is well-suited for pixel-level prediction due to its encoder-decoder structure with skip connections, which help preserve spatial information.

The model takes RGB images as input and outputs a single-channel binary mask. The loss function combines Binary Cross-Entropy (BCE) with Dice loss to balance pixel-wise accuracy and region overlap.

4 Training Setup

The model was trained using the following configuration:

- Input size: 256×256
- Batch size: 8
- Learning rate: 10^{-4}
- Epochs: 15

Training and validation losses were monitored, along with segmentation metrics such as Intersection over Union (IoU) and Dice score.

5 Results

The model achieved the following performance on the test set:

Metric	Value
IoU	0.44
Dice Score	0.56

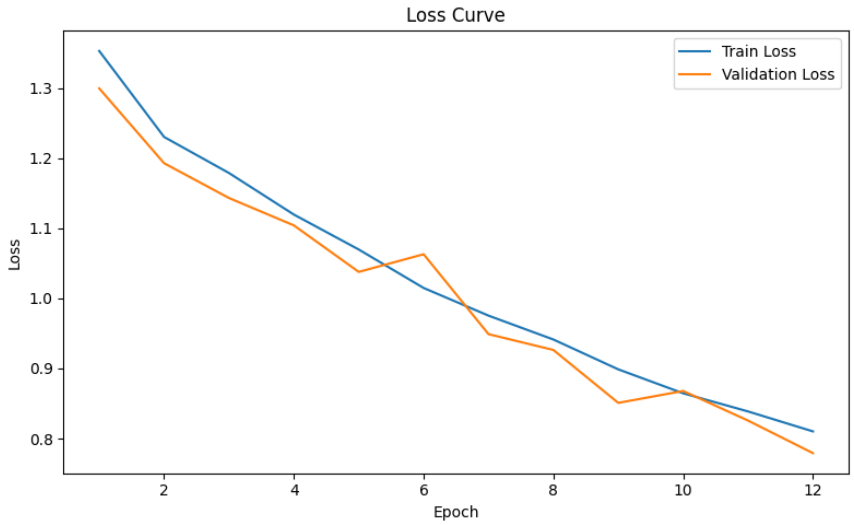

Figure 1: Training and validation loss curves

6 Qualitative Results

Figure below shows example predictions compared to ground truth masks.

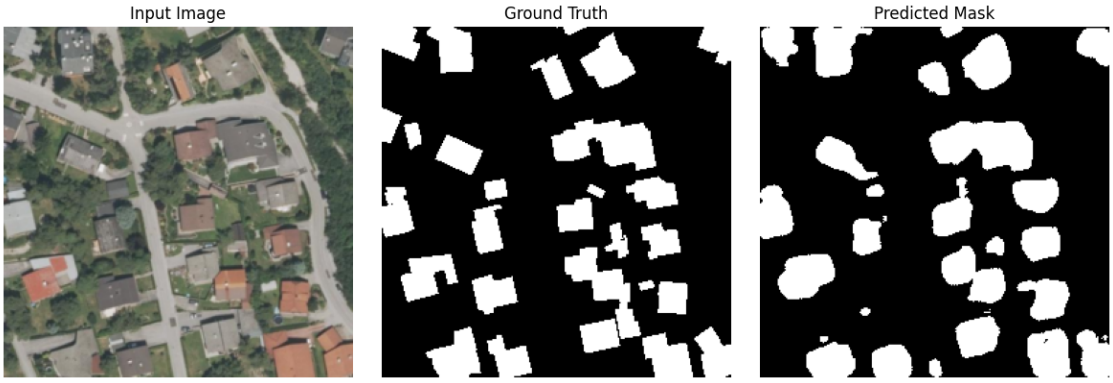

Figure 2: Example segmentation output

7 Discussion

The model successfully identifies building regions but exhibits limitations in boundary precision. Predicted masks tend to be less sharp and may merge nearby structures, indicating limited spatial accuracy.

To investigate whether resolution was a limiting factor, an additional experiment was conducted using higher-resolution inputs (512×512). The expectation was that increased spatial detail would improve segmentation quality, particularly along object boundaries.

However, the results did not show meaningful improvement. In some cases, performance slightly degraded compared to the 256×256 model. The higher-resolution model produced less stable predictions and did not significantly improve IoU or Dice scores.

This behavior can be attributed to several factors:

- **Data limitation**: Higher-resolution inputs increase model complexity and require more data to generalize effectively.
- **Training budget**: The same number of epochs may be insufficient for the model to learn effectively at higher resolution.
- **Model capacity**: The current U-Net architecture may not be expressive enough to fully leverage additional spatial detail.

Overall, this experiment highlights that increasing input resolution alone does not guarantee improved performance, especially when dataset size and training time are constrained.

The primary limitations of the current approach remain:

- Weak supervision from bounding boxes instead of precise pixel-level annotations
- Limited dataset size
- Relatively simple model architecture

8 System Validation and Deployment

To ensure reproducibility and deployment readiness, the system was validated across multiple components.

8.1 API Testing

The segmentation model was exposed through a REST API. The endpoint `/predict-mask` was tested using `curl` and Postman. The API successfully processed input images and returned segmentation outputs.

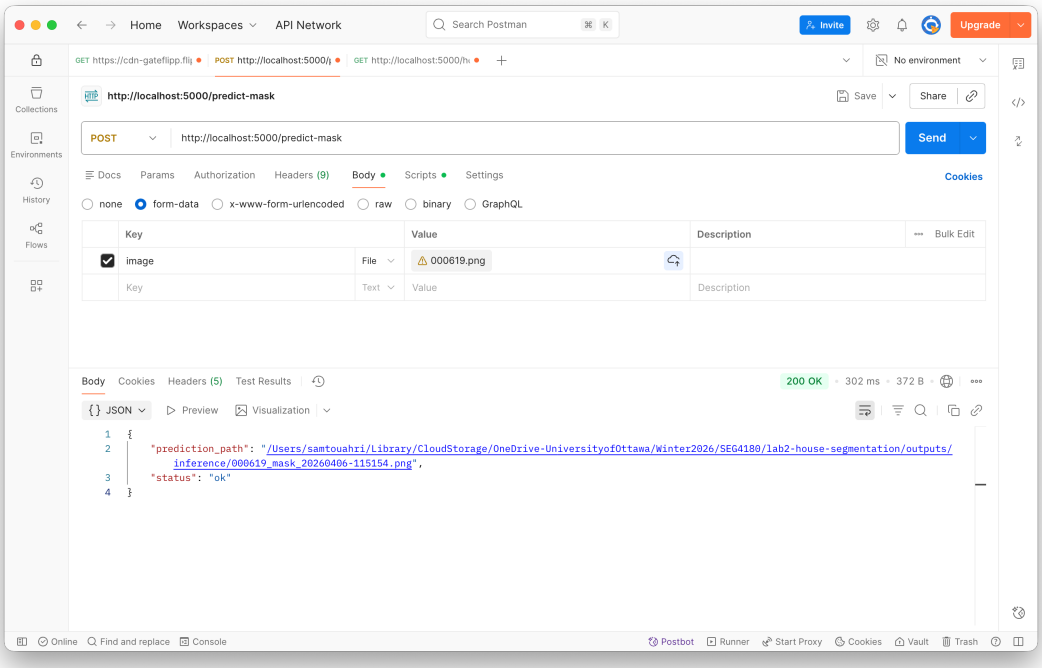

Figure 3: Successful API response for segmentation request

8.2 Docker Deployment

The application was containerized using Docker. The container was successfully built and executed, exposing the API on port 5000.

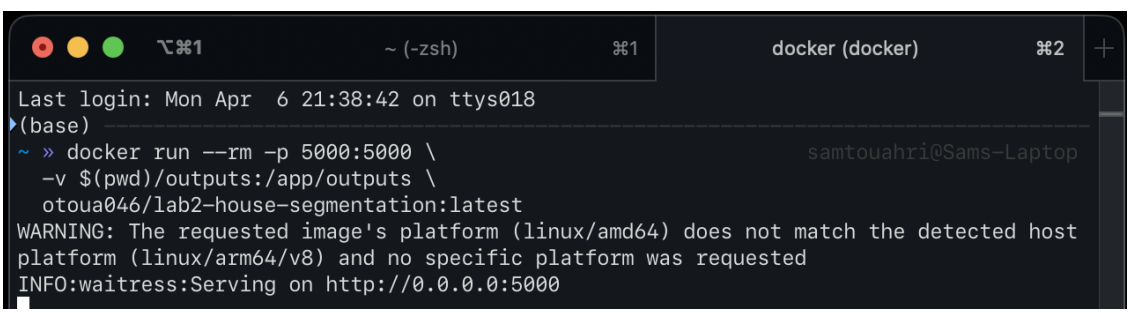

Figure 4: Docker container running the segmentation service

8.3 CI/CD Pipeline

A CI/CD pipeline was implemented using GitHub Actions. The pipeline automatically runs tests, builds the Docker image, and pushes it to Docker Hub.

9 Conclusion

This project demonstrates a complete machine learning workflow from data preprocessing to deployment. While the model achieves reasonable segmentation performance, further improvements could be made with better annotations, larger datasets, and more advanced architectures.